

目标1：利用所学过的知识，完成一个完整的较大的程序



利用面向对象的思维方式，实现面向对象的C++程序！

按照题目的要求，完成具体细节的**设计和实现**。

实例1： 个人银行账户管理系统

// 问题1： 建立一个活期储蓄账户 //

□ 一个活期储蓄账户包括：

□ 信息： 账号 (id) , 余额 (balance) , 年利率 (rate) 等

□ 操作：

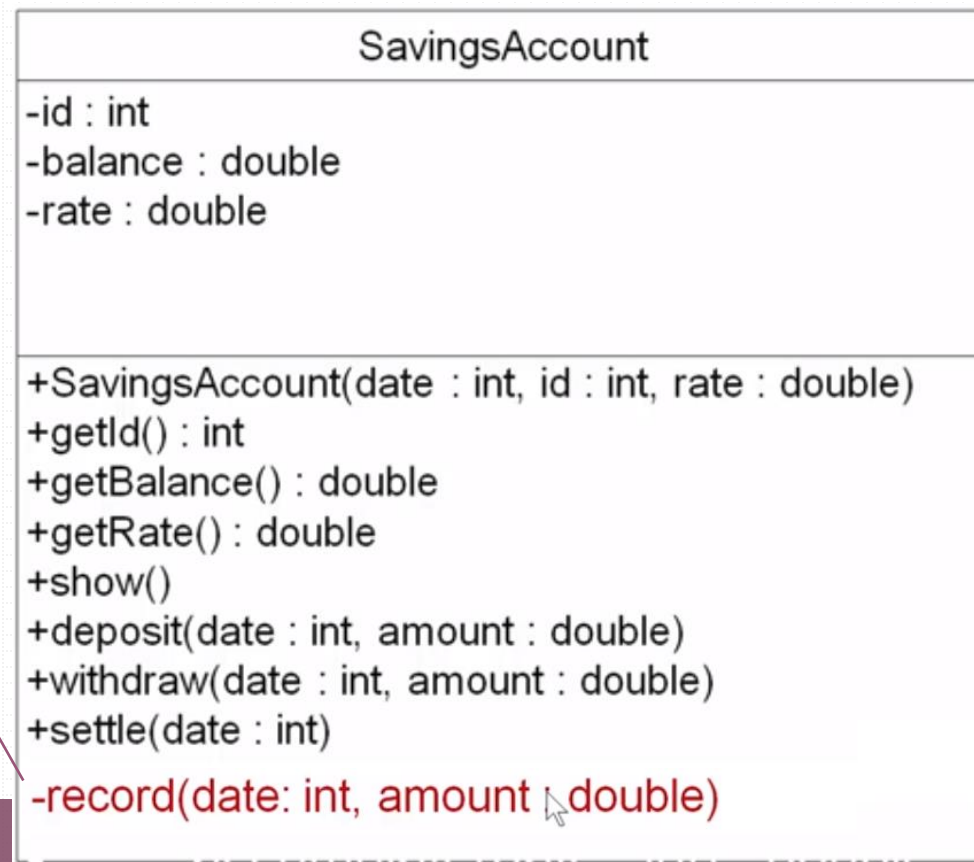
□ 显示账户信息 (show)

□ 存款 (deposit)

□ 取款 (withdraw)

□ 结算利息 (settle) 等

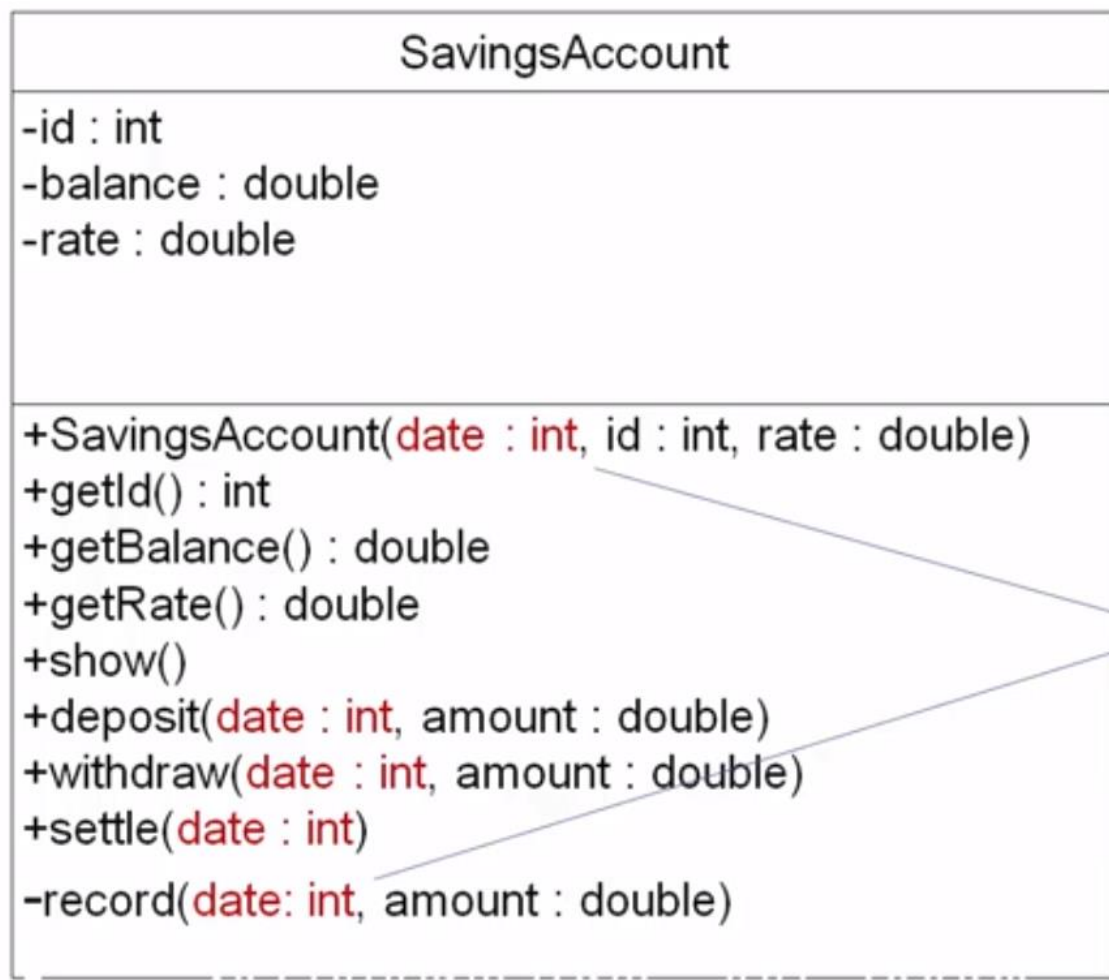
无论是存款、取款还是
结算利息，都需要修改
当前的余额，并且将余
额的变动输出



实例1： 个人银行账户管理系统

// 问题1： 建立一个活期储蓄账户 //

□ 一个活期储蓄账户包括：



□ 为简便起见，该类中的所有日期均用一个整数来表示

□ 该整数是一个以日为单位的相对日期，例如如果以开户日为1，那么开户日后的第3天就用4来表示

□ 这样通过将两个日期相减就可以得到两个日期相差天数，在计算利息时非常方便

实例1： 个人银行账户管理系统

// 问题1： 建立一个活期储蓄账户 //

□ 利息的计算1： 设计难点

□ 由于账户的余额是不断变化的， 因此：

□ 余额 * 年利率 = 年利？ **No!**

□ 一年当中每天的余额累计起来/一年的总天数 = 日均余额。

□ 日均余额 * 年利率 = 年利。 **Yes!**

□ 例如， 如果年利率是1.5%， 某账户第5天存入5000元， 第45天存入5500元， 第90天结算利息。

□ 它第5天到第45天之间的余额为5000元 （持续40天）

□ 第45天到第90天之间的余额为10500元 （持续45天）

□ 90天的利息是 $(40 * 5000 + 45 * 10500) * 1.5\% / 365 = 27.64$ 元

实例1： 个人银行账户管理系统

// 问题1： 建立一个活期储蓄账户 //

□ 利息的计算2

□ 为了计算余额的按日累积值，SavingsAccount引入了：

□ 私有数据成员lastDate: 存储上一次余额变动的日期。

□ 私有数据成员accumulation: 存储上次计算利息以后直到最近一次余额变动时余额按日累加的值。

□ 私有数据函数accumulate: 计算截至指定日期的账户余额按日累积值。

□ **当余额变动时**，需要做的是将变动前的余额与该余额所持续的天数相乘，累加到accumulation中，再修改lastDate。

□ $\text{Accumulation} + \text{balance} * (\text{date} - \text{lastDate});$

实例1： 个人银行账户管理系统

// 问题1： 建立一个活期储蓄账户 //

□ SavingsAccount

SavingsAccount
-id : int -balance : double -rate : double -lastDate : int -accumulation : double
-record(date: int, amount : double) -accumulate(date : int) : double +SavingsAccount(date : int, id : int, rate : double) +getId() : int +getBalance() : double +getRate() : double +show() +deposit(date : int, amount : double) +withdraw(date : int, amount : double) +settle(date : int)

实例1： 个人银行账户管理系统

❑ 细节实现： 四舍五入的处理

❑ 如何保留小数点后两位

```
void SavingsAccount::record(int date, double amount) {  
    accumulation = accumulate(date);  
    lastDate = date;  
    amount = floor(amount * 100 + 0.5) / 100; //保留小数点后两位  
    balance += amount;  
    total += amount;  
    cout << date << "\t#" << id << "\t" << amount << "\t" << balance << endl;  
}
```

❑ floor函数：

- ❑ 向下取整函数（在数学上称为高斯函数）
- ❑ 用来得到不大于一个数的最大整数
- ❑ 声明在头文件cmath中。

❑ 对一个数x做四舍五入取整：floor(x+0.5)

❑ floor(amount*100+0.5)/100： 保留小数点后两位

❑ 补充： Ceil为向上取整函数

实例1： 个人银行账户管理系统

// 问题2： 完善一个活期储蓄账户 //

- ❑ **查看当前所有账户的总金额是一个经常性的需求**，可是把“全体账户总金额”做为类的普通成员加入，就意味着在每一个账户实例中都有一份copy，不仅仅是浪费存储空间，更困扰的是需要小心维护保持数据的一致性。应该怎么做？有什么好办法呢？
- ❑ 用一个整数来**表示银行账号**，这并不是完美的方案，例如，如果银行账号以“0”开头，或账号超过整数的表示范围，或账号中包括其他字符，这种表示都不能胜任。

实例1： 个人银行账户管理系统

// 问题2： 完善一个活期储蓄账户 //

- ❑ 主程序创建的两个账户为两个独立的变量，只能用名字去引用它们，在主程序末尾分别对两个账户进行结算（settle）和显示（show）时，需要将几乎相同的代码书写两遍，如果账户数量增多将会带来更大麻烦。
- ❑ 日期用一个整数表示，这样计算两个日期相距天数时非常方便，但这种表示很不直观，对用户很不友好。

实例1： 个人银行账户管理系统

// 问题2： 完善一个活期储蓄账户 //

- ❑ 总账户金额的存储和维护
- ❑ 日期的友好处理
- ❑ 账号的合理表示

SavingsAccount
-id : string -balance : double -rate : double -lastDate : Date -accumulation : double <u>-total : double</u>
-record(date: Date, amount : double, desc : string) -<<const>> -accumulate(date : Date) : double +SavingsAccount(date : Date, id : int, rate : double) -<<const>> +getId() : int -<<const>> +getBalance() : double -<<const>> +getRate() : double -<<const>> +show() +deposit(date : Date, amount : double, desc : string) +withdraw(date : Date, amount : double, desc : string) +settle(date : Date) -<<static>> +getTotal() : double

为SavingsAccount类增加一个静态数据成员total，其好处在于全体账户对象共享一份。既能节省存储空间、又无需担心数据一致性的问题

这些不需要改变对象状态的成员函数，我们可以将其声明为常成员函数，用const修饰。

并且相应的，为其增加静态成员函数getTotal用来对其进行访问。

实例1： 个人银行账户管理系统

// 日期的友好处理

- ❑ 日期用一个整数来表示
 - ❑ 这样计算两个日期相距天数时非常方便
 - ❑ 这种表示**很不直观，对用户很不友好**
- ❑ 日期可以用一个类来表示
 - ❑ 内含年、月、日三个数据成员
 - ❑ 但这样对计算**两个日期相差天数带来了麻烦**
- ❑ 计算日期间相差的天数
 - ❑ 可以选取一个比较规整的基准日期，在构造日期对象将该日期到这个基准日期的相对天数计算出来，我们将这个相对天数称为 **“相对日期”**
 - ❑ 这样在计算两个日期的相差天数时，只需将二者的相对日期相减即可。

实例1： 个人银行账户管理系统

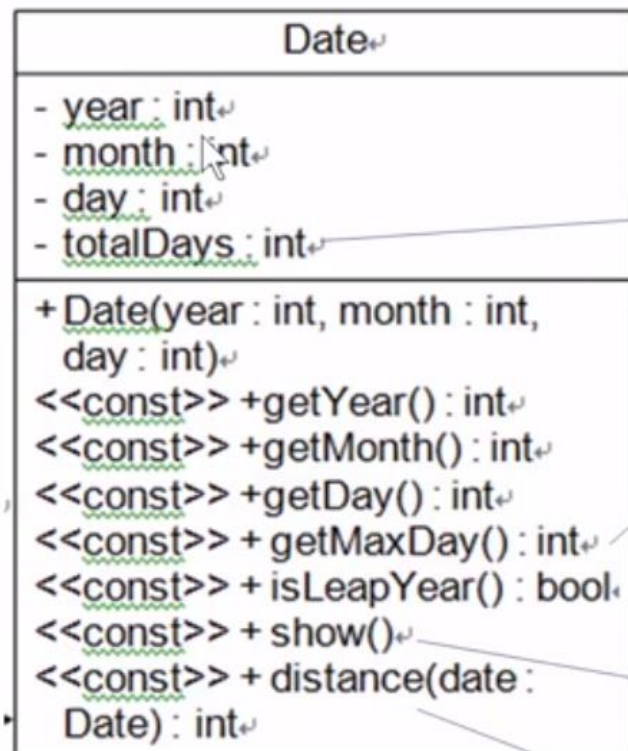
// 日期的友好处理

- 将公元元年1月1日作为公共的基准日期，将y年m月d日相距这一天的天数记为 $f(y/m/d, 1/1/1)$ ，这样可以将其**分解为三部分**：
 - $f(y/m/d, 1/1/1) = f(y/1/1, 1/1/1) + f(y/m/1, y/1/1) + f(y/m/d, y/m/1)$
 - 其中，
 - $f(y/1/1, 1/1/1) = 365 * (y-1) + (y-1)/4 - (y-1)/100 + (y-1)/400$
 - $f(y/m/d, y/m/1) = d - 1$
- 由于每月天数不同，因此 $f(y/m/1, y/1/1)$ 难以表示为一个统一的公式
 - 平年指定月份的1日与1月1日相差天数可以由月份m唯一确定，因此可以**把每月1日到1月1日的天数放在一个数组里，计算时只要查询该数组**，便可以得到 $f(y/m/1, y/1/1)$ 的值。
 - 闰年，仍可以通过数组查询，只需要 $m > 2$ 时将查到的值加1。该值只依赖m和y，我们将其记为 $g(m, y)$
 - $f(y/m/d, 1/1/1) + 1 = 365 * (y-1) + (y-1)/4 - (y-1)/100 + (y-1)/400 + g(m, y) + d$

实例1： 个人银行账户管理系统

// 类完善后的UML图

□ Date类的UML图



totalDays表示这一天的相对日期

得到当前月的天数的getMaxDay函数

用来判断当前年是否为闰年的isLeapYear函数

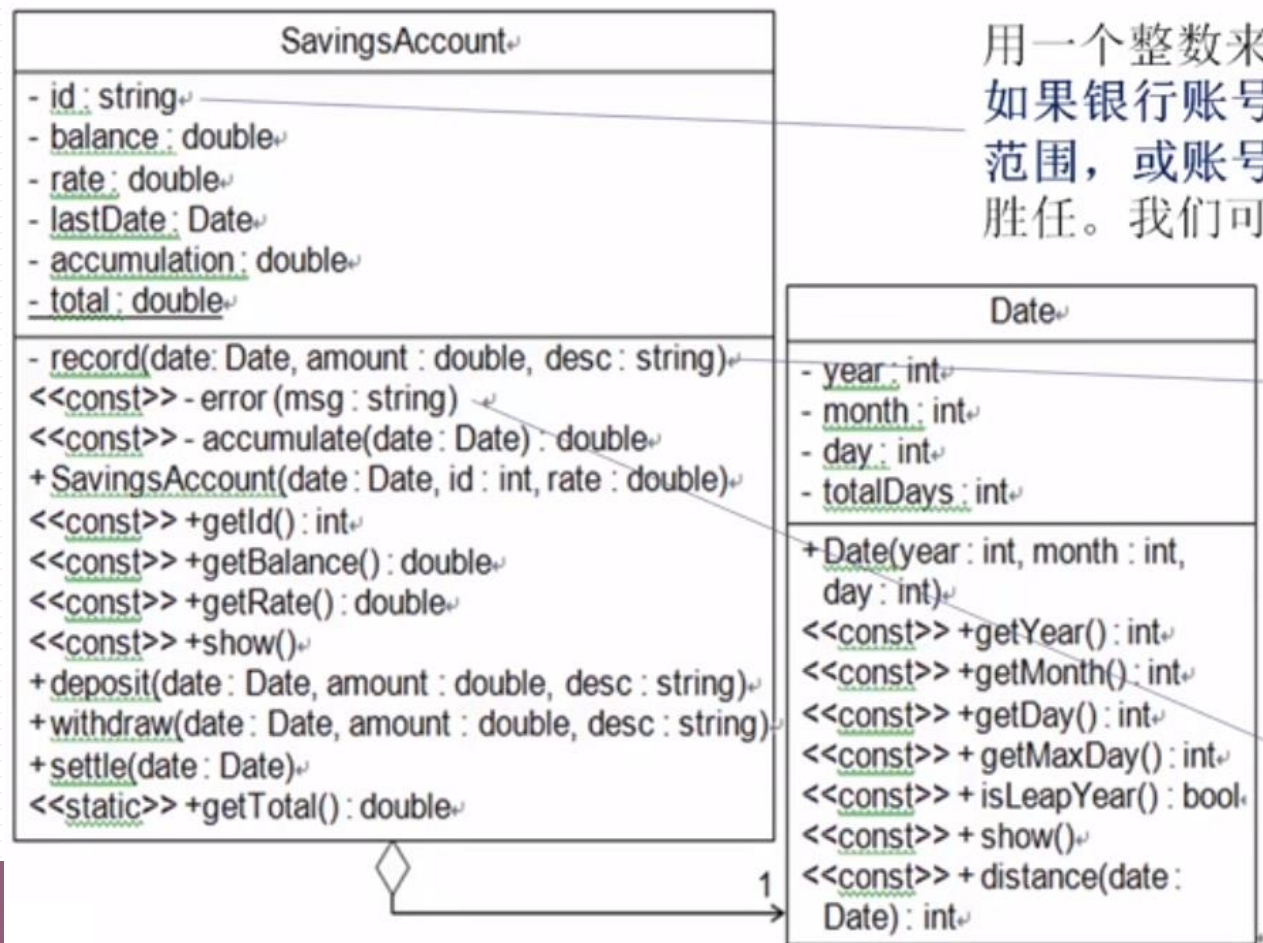
用来将当前日期输出的show函数

用来判断当前日期与指定日期相差天数的distance函数

实例1： 个人银行账户管理系统

// 类完善后的UML图

□ 类设计的UML图



用一个整数来表示银行账号，但这并不完美，例如，如果银行账号以“0”开头，或账号超过整数的表示范围，或账号中包括其它字符，这种表示方式都不能胜任。我们可以改用字符串来表示银行账号

账目列表，每笔账目都没有说明，使用字符串可以为各笔账目增加说明文字

专门增加了一个用来报告错误的函数，当其它函数需要输出错误信息时，直接把信息以字符串形式传递给该函数即可，简化了错误信息的输出。

实例1： 个人银行账户管理系统

// 利用数组避免代码冗余 //

- 之前主程序创建的两个账户为两个独立的变量：
 - 只能用名字引用对象，在主程序末尾对两个账户进行结算（settle）和显示（show）时需要将几乎相同的代码书写两遍
 - 如果账户数量增多将会带来麻烦
- 将多个账户组织在一个数组中，这样对各个账户所做的一致操作放在循环中，避免了代码的冗余

实例1： 个人银行账户管理系统

// 代码提示细节- exit函数

```
//date.cpp
#include "date.h"
#include <iostream>
#include <cstdlib> //exit函数，该函数的原型声明在cstdlib中
using namespace std;

namespace { //namespace使下面的定义只在当前文件中有效
//存储平年中某个月1日之前有多少天，为便于getMaxDay函数的实现，该数组多出一项
const int DAYS_BEFORE_MONTH[] = { 0, 31, 59, 90, 120, 151, 181, 212, 243, 273, 304, 334, 365 };
}

Date::Date(int year, int month, int day) : year(year), month(month), day(day) {
if (day <= 0 || day > getMaxDay()) {
cout << "Invalid date: ";
show();
cout << endl;
exit(1); //用来中止当前程序的执行，并将一个整数返回给系统，通常，0表示正常退出，1表示异常退出
}
```